

Technical Practice Report

EJavdge Client Library and IntelliJ IDEA Plugin

Ejudge Integration, DSL Design, Testing and Plugin Publication

University and Faculty: Shenzhen MSU-BIT University

Programme and Class: Mathematic and Applied Mathematic

Student Names: Emil Kurtametov, Tu Jingrui

Supervisor: Aleksei Demin

Start Date and Practice Period: 28.07.2026---07.10.2026

Contents

1 Introduction.....	3
2 EJavdge and Plugin Development.....	4
3 DSL and Replaceable Implementations	6
4 Main User Workflows	7
5 IntelliJ IDEA Integration	11
6 Testing and Verification	13
7 Reliability and Current Limitations.....	14
8 Results and Further Development	16
References.....	17
Appendix A Versions and Commands	18
Appendix B Example Source Program.....	19

1 Introduction

1.1 Original Practice Goal

The original goal of the practice was to develop a mechanism that would allow users to work with the ejudge system conveniently directly from IntelliJ IDEA. The task was therefore to provide a way to perform the main actions with ejudge problems without constantly switching between the IDE and a browser.

1.2 Tasks and Work Plan

According to the original goal, the work was organized around the following tasks: develop a mechanism for working with ejudge directly from IntelliJ IDEA; implement problem viewing and retrieval of problem data; implement retrieval of attachments; implement local checking of source code against available samples; implement source submission and retrieval of the checking result; provide clear diagnostics for source-file and server-interaction errors; integrate the client logic with the IntelliJ IDEA interface; and additionally verify the implementation with automated tests.

1.3 Project Scope

EJavdge is a Java client for working with an existing ejudge service. It is not the ejudge server and does not replace the official judging mechanism. The IntelliJ IDEA plugin provides the user interface to the client functionality.

2 EJavdge and Plugin Development

2.1 Problem Statement and Engineering Decisions

For Java learners and contest participants, the system reduces IDE/browser switching through composable actions for problem viewing, attachment retrieval, sample checking, source submission and report viewing. Remote ejudge remains the authoritative source of official judging results. [1][2]

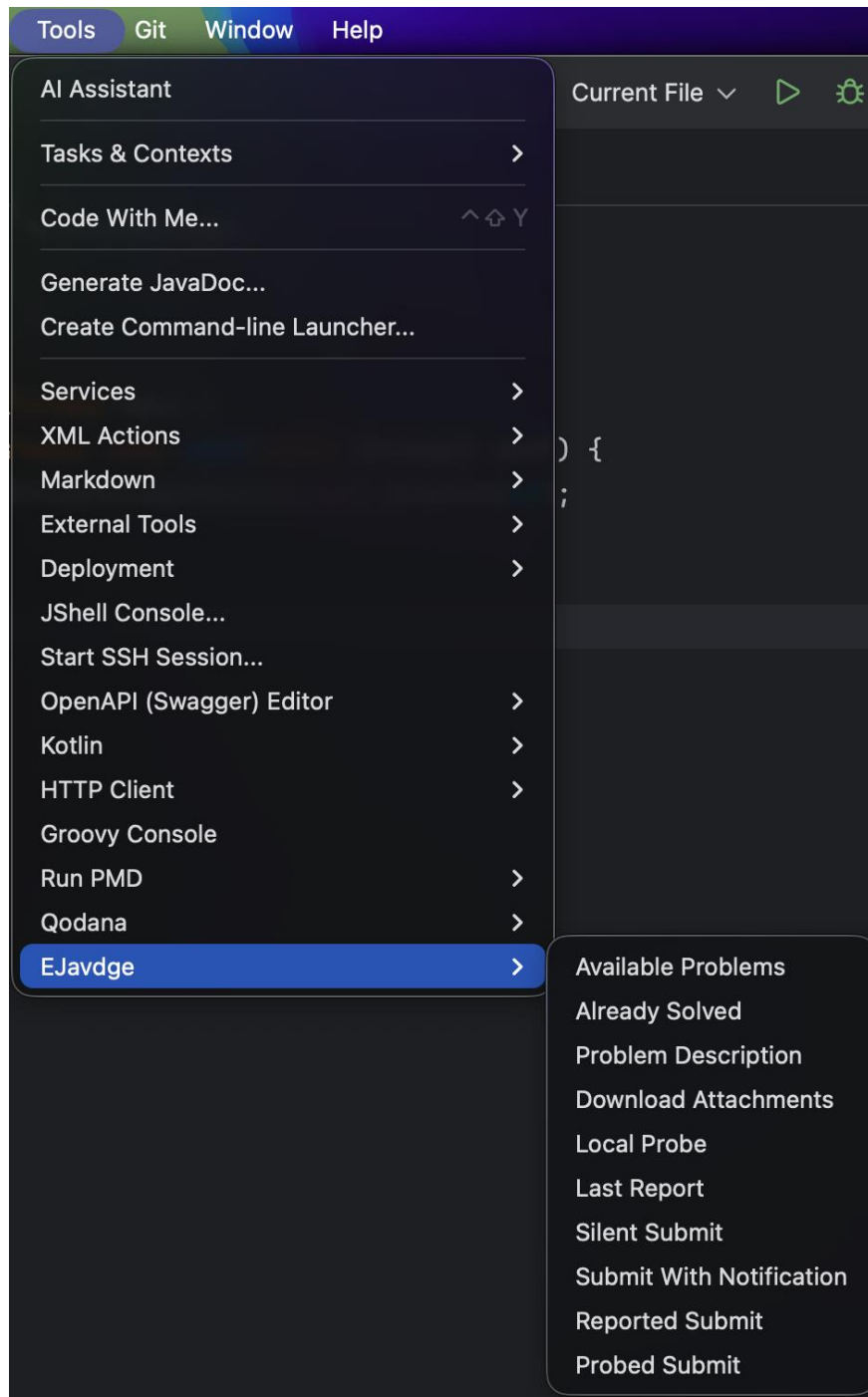


Figure 1. EJavdge tools available in IntelliJ IDEA.

2.2 Main Development Results

The development resulted in a Java client EJavdge, an IntelliJ IDEA plugin, IDE actions for the main problem and submission workflows, problem and attachment retrieval, local sample checking, submission and report polling, retrieval of the latest report, password storage through IntelliJ IDEA PasswordSafe, a set of automated tests and fixtures, and publication of the plugin on JetBrains Marketplace.

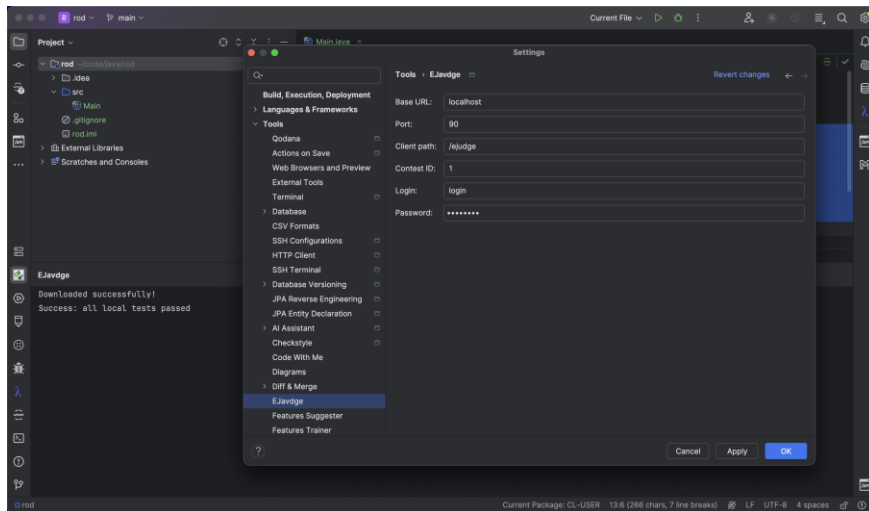


Figure 2. EJavdge plugin settings with contest credentials.

2.3 Architecture and Object Composition

EJavdge is built from small semantic objects. Interfaces such as Text, Bytes, Items, Effect, Context and Media separate values, collections, contexts and side effects. Constructors assemble dependencies, while content(), run() and perform() explicitly denote obtaining a result or executing an action. For example, ContestResource combines WebDriver, Location and Session; Context adds parameters to Media; and SilentSubmit obtains the problem identifier and language, builds the solution context and performs the submission. This style was chosen by the authors to compose client workflows and make infrastructure replaceable in tests.

3 DSL and Replaceable Implementations

3.1 Web DSL and HTTP Specification

Configuration and authentication are provided as part of the client workflow. The core reads LOGIN, PASSWORD, CONTEST_ID, BASE_URL, PORT and CLIENT_PATH from .env by default. Location combines host, port and path; Credentials represents account and contest context; Session obtains SID and EJSID from the login response for reuse in contest-page and submission requests. [1]

3.2 HTTP Transport and Response Processing

The web layer provides a DSL built on top of the HTTP specification. It represents methods, resources, headers, bodies, forms and multipart data separately. A user workflow first constructs an HTTP request specification and then passes it to a concrete execution mechanism. JdkSocket is the standard transport implementation in the current project, but the DSL itself is not conceptually tied to JdkSocket: another implementation capable of executing the constructed HTTP specification can be substituted.

HTTP responses are separated into headers and body. HttpResponse identifies the boundary of the header section of the stream, after which BodyOf exposes the body stream. This avoids requiring the complete socket stream to be read before the header boundary is known.

3.3 DOM DSL and XPath 2.0

The DOM layer follows the same principle. It provides a DSL built on the XPath 2.0 language rather than directly on a particular parsing library. XmlEngine defines the contract of a mechanism that applies an XPath 2.0 query to a supplied XML document. JsoupWithSaxon is the standard XmlEngine implementation and uses Jsoup and Saxon to prepare the XML and execute XPath 2.0. The implementation can be replaced by another XmlEngine implementation that provides the required XPath 2.0 execution. Thus Jsoup and Saxon are infrastructure of the current implementation, not part of the DOM DSL.

3.4 Linking Source Files to Problems and Languages

The source file contains a string marker `// problem: <SHORT_NAME>` used to identify the problem. The short name starts with an ASCII letter and is followed by letters or digits. A language can be specified with the marker `// language: <ID_IN_SELECTOR>`, which identifies the language position in the problem selector.

If the problem marker is missing, the user receives: Error: There is no problem marker (comment like ``// problem: <SHORT_NAME>`` is required). If the language marker is missing, the user receives: Error: There is no language marker (comment like ``// language: <ID_IN_SELECTOR>`` is required). The markers form part of the source-file DSL and connect the source file with the problem and language without a separate IDE configuration.

4 Main User Workflows

4.1 Problem Viewing and Attachment Retrieval

The client obtains ejudge pages through the Web DSL, while the DOM DSL performs XPath 2.0 selections from which problem, language, run and report objects are constructed. Attachments can then be retrieved and stored locally.

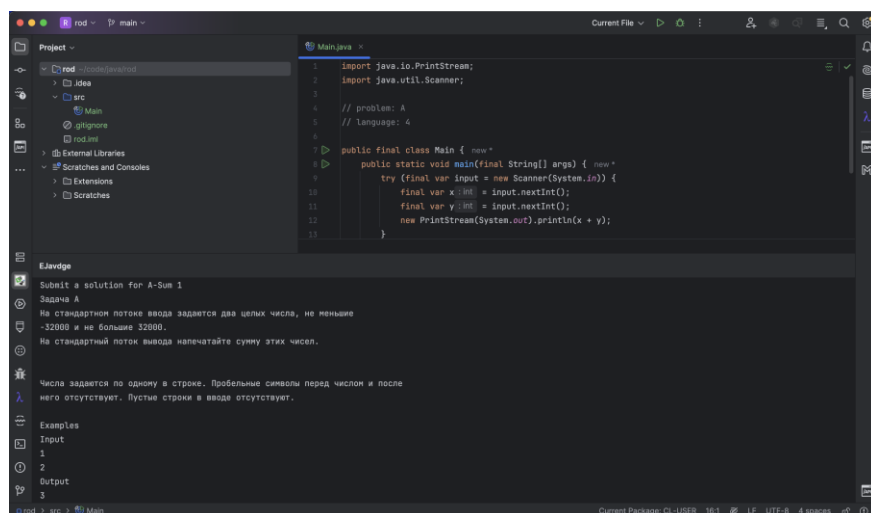


Figure 3. Problem statement displayed in IntelliJ IDEA.

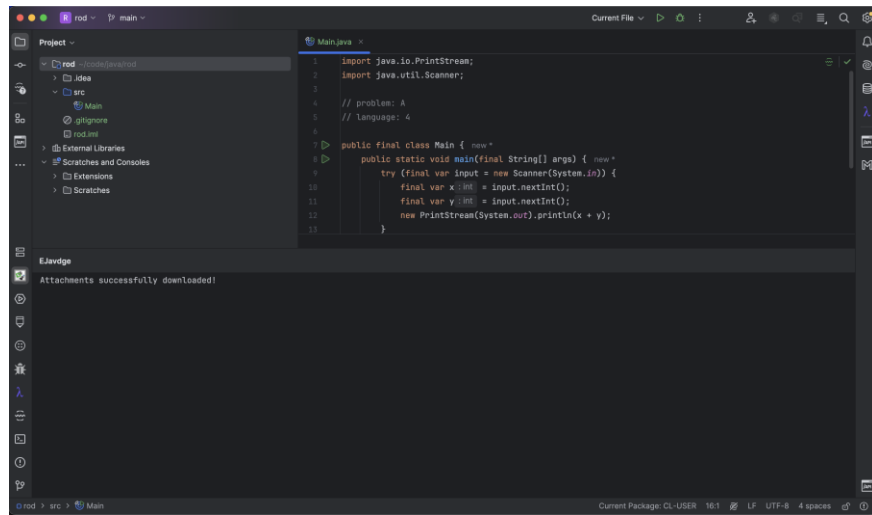


Figure 4. Downloading problem attachments from ejudge.

4.2 Local Sample Checking

JavaProgram writes the source file to a temporary **.java** file and runs it in **Java source-file mode**. Input is passed through **stdin**, process output is read, the exit code is checked, and **VerdictBySamples** compares actual and expected output. Successful samples do not replace hidden **ejudge** tests; a special judge is not implemented by the current mechanism.

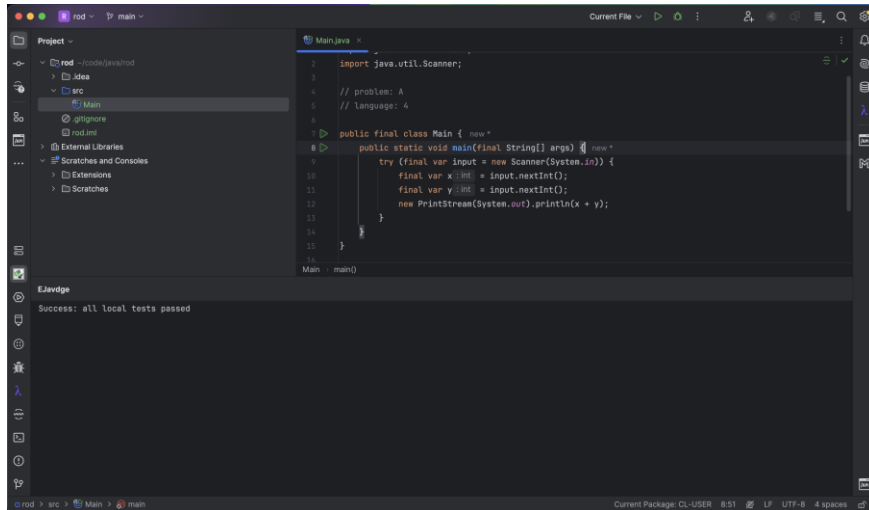


Figure 5. Successful local sample check.

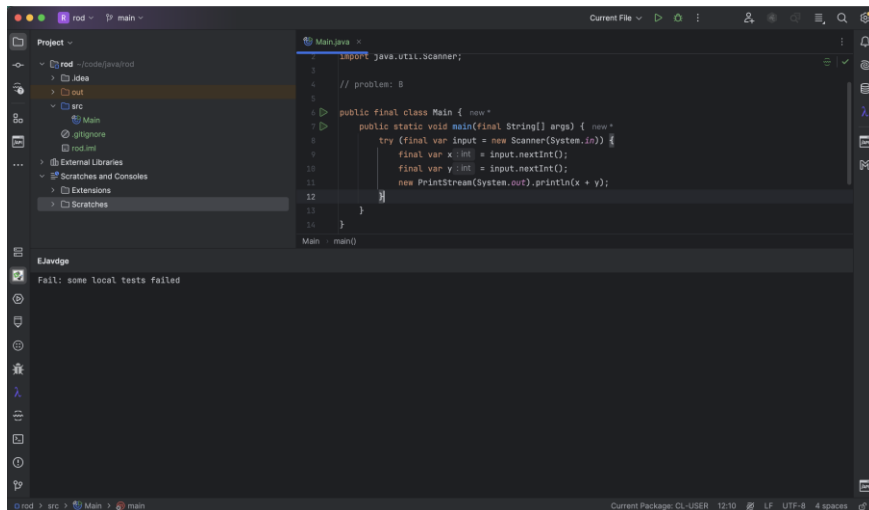


Figure 6. Failed local test result with diagnostic output.

4.3 Submission and Report Retrieval

SilentSubmit connects the source file with the problem identifier and selected language, constructs the solution context and performs the submission. Higher-level compositions add local checking, user notification and waiting for the report. StatusInJson obtains the report status, ReportReadiness determines whether the report is available, WithPolling repeats the request with a delay, and LastReport selects the latest available report.

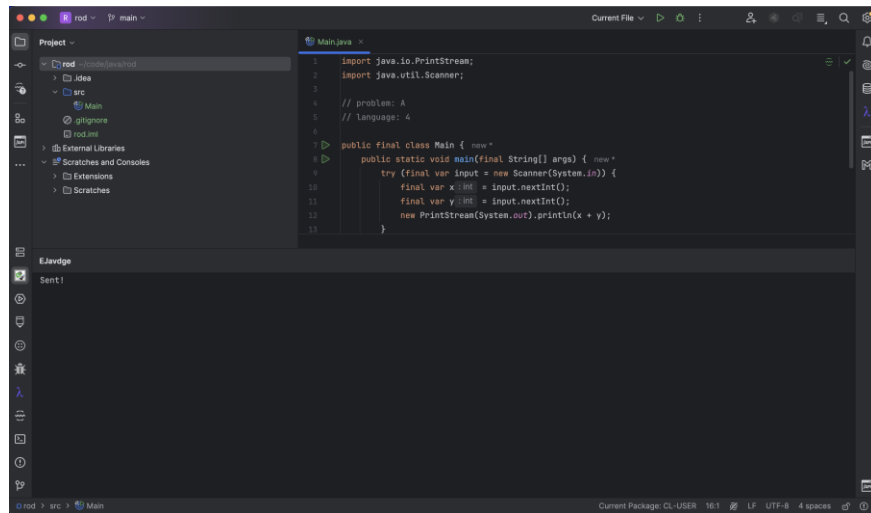


Figure 7. Submitting a solution to ejudge.

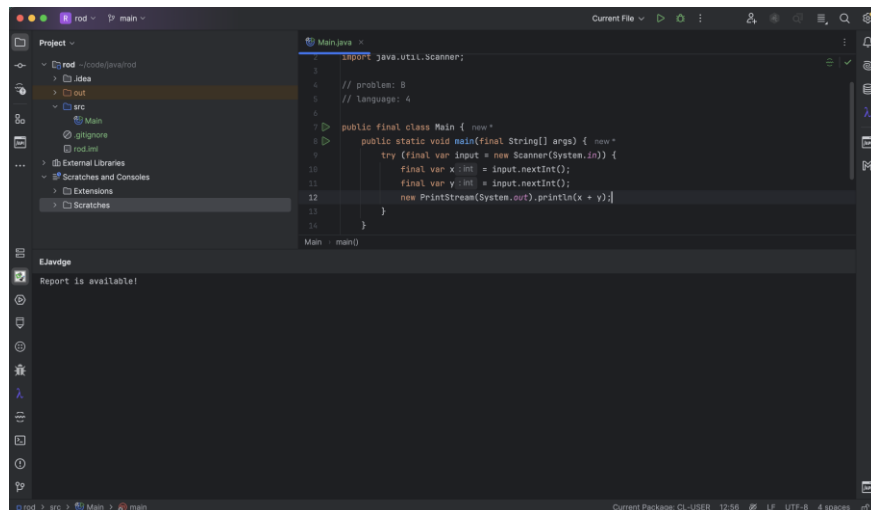


Figure 8. Notification that the report is available.

5 IntelliJ IDEA Integration

5.1 Actions and Output Adapters

The IntelliJ plugin registers ten user actions for the main contest workflows. plugin.xml also registers the Tools menu and toolbar entries, the settings page and the EJavdge tool window. CurrentFile obtains the selected file from the event context and ProjectOf obtains the project. After the file is saved, an action passes the source and project parameters to the core.

5.2 Settings and Credentials

EjState persists the host, port, path, contest ID and login name, but not the password. SaveSettings passes the password to SettingsPassword, which uses IntelliJ IDEA PasswordSafe. This was an engineering decision made by the authors during development.

5.3 Build and Publication

The current plugin configuration uses JDK 17, Gradle 8.9 and IntelliJ IDEA 2023.2.6, with a declared platform build range of 232–242.*. The configuration contains buildPlugin, signPlugin and publishPlugin.

- **The plugin was published on JetBrains Marketplace, which is now the primary end-user installation channel. Source builds and manual ZIP installation remain useful for development, debugging and reproducible packaging.**

The plugin uses EJavdge core v1.0.1 from JitPack, so an end user of the published plugin does not need to build the core library separately.

- The publication route changes the practical installation workflow: users install and update the plugin through JetBrains Marketplace, while source builds remain relevant for development and reproduction. [7][8]
- The published plugin therefore provides a single user-facing installation path while preserving the source build as a development and verification mechanism.

The plugin repository can still be built with the included Gradle wrapper;
`buildPlugin` produces the installable ZIP for local development and reproduction.

6 Testing and Verification

6.1 Automated Tests

Automated tests were not part of the original practice requirement. They were introduced by the authors as an independent engineering decision for controlling the quality of their implementation. The core project uses Surefire for unit tests and Failsafe for integration tests; test dependencies allow network and DOM mechanisms to be replaced by fixtures.

6.2 Coverage and Static Analysis

Level	Research Scope	Evidence
Static analysis	Modules, interfaces and configuration	Source and build files
Unit tests	Values, protocol and workflows	Surefire logs
Integration tests	Page fixtures, files and subprocesses	Failsafe logs
Plugin build and publication	IDE artifact and release workflow	Gradle configuration, build artifact and Marketplace publication

JaCoCo recorded 1,907 covered lines out of 2,056, corresponding to 92.75% line coverage. PMD reported no violations in the verification target. These metrics describe the quality checks chosen by the authors and were not requirements of the original practice task.

6.3 Verification Boundaries

The automated tests use local fixtures and do not prove behavior against every possible real contest server. No independent Windows/Linux/macOS test matrix was established, so full cross-platform compatibility is not claimed.

7 Reliability and Current Limitations

Test scope	Tests	Failures / Errors	Result
Surefire unit tests	818	0 / 0	Passed
Failsafe integration tests	202	0 / 0	Passed
Combined automated tests	1020	0 / 0	Passed
JaCoCo line coverage	92.75 %	90% minimum	Passed

7.1 HTTP and Security

The standard JDK Socket transport does not provide TLS. WithLogsDriver can record raw HTTP data at TRACE level, so sensitive data, session cookies and source code may require redaction before such logging is enabled in a sensitive environment. PasswordSafe addresses password storage in the IDE, but not transport security or log redaction.

7.2 Local Program Execution

JavaProgram launches a local Java process without a dedicated sandbox model, a separate output-size limit or full resource isolation. The current model is therefore intended for controlled educational samples rather than public execution of arbitrary untrusted code.

7.3 File System and Cross-Platform Risks

Saving attachments requires control of normalized paths, target-directory boundaries and duplicate-name policy. Test fixtures also contain line-ending-

sensitive constructions; this should be treated as a portability risk rather than a reproduced Windows failure.

8 Results and Further Development

8.1 Final Results

The main goal of the practice was achieved: a mechanism was developed that allows users to work with ejudge directly from IntelliJ IDEA. The implemented workflows provide problem viewing, attachment retrieval, local sample checking, source submission and result viewing.

An additional result was the EJavdge client library developed by the authors. The library, the DSLs, the object composition, the replaceable infrastructure and the automated test suite were not prescribed by the original task; they were engineering decisions made during development.

The plugin has been published on JetBrains Marketplace, which is now the primary user-facing installation channel.

8.2 Possible Improvements

Possible improvements include a TLS transport implementation; redaction of sensitive data in TRACE logs; time, memory and output limits for local processes; path and duplicate-name controls for downloaded attachments; configurable polling and timeout; broader verification of IntelliJ Platform versions; a cross-platform test matrix; expanded DOM fixtures for changes in ejudge pages; and background IntelliJ tasks with progress reporting and cancellation for long-running operations.

8.3 Conclusion

The practice resulted in an integration of ejudge with IntelliJ IDEA. The main result is a working user-facing mechanism that allows the main actions with problems and submissions to be performed directly from the IDE.

References

Source at the pinned revision[1] F1rrock. EJavdge client source and README. GitHub repository, commit a5f7d96a5b5084be7de2a86e02f9f232fdd1d023.

Source at the pinned revision[2] F1rrock. EJavdge for IntelliJ IDEA source and README. GitHub repository, commit 71547c985929eef851f6d8a01c73a8c25ce7a2f.

[3] JetBrains. Persisting Sensitive Data. IntelliJ Platform Plugin SDK.Sensitive data storage documentation

[4] Apache Software Foundation. Maven Failsafe Plugin Introduction.Integration test lifecycle documentation

[5] Oracle. Java SE Formatter. Java API Specification.Formatting and platform line-ending specification

[6] JetBrains. Background Processes. IntelliJ Platform Plugin SDK.Background task and progress documentation

[7] JetBrains. About JetBrains Marketplace.

[8] JetBrains. Publishing a Plugin.

[9] F1rrock. EJavdge for IntelliJ IDEA: complete plugin screenshots and user scenarios. GitHub repository, docs/screenshots.<https://github.com/F1rrock/ejavdge-intellij/tree/main/docs/screenshots>

Project-derived statements in this report are based on the project source archives and generated verification artifacts. External documentation is used to describe platform mechanisms and publication procedures; it is not treated as evidence of live ejudge acceptance.

Appendix A Versions and Commands

A.1 Pinned Revisions

Full core-library commit hash

a5f7d96a5b5084be7de2a86e02f9f232fdd1d023

Full plugin commit hash

71547c985929eef851f6d8a01c73a8c25ce7a2f

A.2 Core Verification Commands

The core verification can be reproduced in the corresponding source directory with JDK 17 available in PATH and Maven installed. The recorded target artifacts were produced by Maven Surefire/Failsafe, JaCoCo and PMD.

```
mvn -B test
```

```
mvn -B verify
```

The current source-of-truth verification is the generated target state: 818 unit tests and 202 integration tests, all passing. The automated verification was added by the authors as an engineering quality measure.

A.3 Plugin Build and Publication

The plugin uses JDK 17, Gradle 8.9 and IntelliJ IDEA 2023.2.6 as its build target, declares compatibility with builds 232–242.* and resolves EJavdge core v1.0.1 from JitPack. buildPlugin produces a local installable ZIP; signPlugin and publishPlugin support the release workflow. The published plugin is installed through JetBrains Marketplace.

Appendix B Example Source Program

B.1 Minimal Solution Example

```
// problem: A
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        try (Scanner in = new Scanner(System.in)) {
            System.out.println(in.nextLong()
                               + in.nextLong());
        }
    }
}
```

For input 2 3, the expected output is 5. Replace A with the actual identifier in a test contest. This example illustrates the local sample workflow only and is not evidence of online acceptance.

B.2 Engineering Decisions

The separate core library, IntelliJ plugin, DSL abstractions, replaceable infrastructure implementations and automated tests are engineering decisions made by the authors during development rather than requirements of the original practice task.

B.3 Future Acceptance Records

For future live acceptance testing, record the operating system, JDK, core and plugin versions, server version, problem, language, submission identifier, input and output, expected and actual status, and log location. Sensitive account and session information should be redacted.